Qwen

Architecting Neuro-Adaptive Horror: A Multi-Layer Contract System for Safe and Consistent BCI Integration in the vm-constellation

The development of Brain-Computer Interface (BCI) device adapters for the Horror$Place vm-constellation requires a foundational architectural framework that prioritizes system coherence, user safety, and cross-device interoperability. This report outlines a comprehensive research framework designed to guide the creation of these adapters, ensuring they integrate seamlessly with existing runtime modules—including `H.`, `BCI.`, `Director.`, and `Policy.DeadLedger`—while enforcing safety and comfort as non-negotiable constraints . The proposed architecture is built upon a multi-layered contract system that governs the flow of information from raw physiological signals to runtime game logic. It establishes a clear separation between the scientific processing of biometric data, which occurs within dedicated labs, and the governance of horror content, which is managed by the Dead-Ledger protocol. This approach allows for innovation in signal mapping and sensor technology without compromising the stability or security of the core gaming experience. The framework treats safety and user comfort as hard requirements, enforced through cryptographic proofs and automated system responses, while permitting creative narrative modulation only within predefined, safe boundaries . To achieve consistent behavior across a diverse range of BCI hardware, the system mandates strict adherence to a canonical payload schema and intensity mode rules, allowing for controlled experimentation only under explicitly versioned contracts . This report details the components of this framework, including its tiered data abstraction model, cryptographic attestation protocols, canonical runtime interfaces, and detailed implementation blueprints for device adapters and associated telemetry pipelines.

The Tiered Data Abstraction Model for Secure BCI Signal Handling

A cornerstone of the proposed BCI integration framework is a strictly defined, three-tiered data abstraction model. This model is designed to manage the sensitivity and complexity of raw physiological data by quarantining it at its source while progressively transforming it into standardized, commitment-friendly abstractions suitable for governance and runtime execution. This layered approach directly addresses the need to leverage neuro-state information for adaptive horror content generation without exposing personally identifiable information (PII), raw biometric streams, or device-specific proprietary details to the broader system. The ownership, visibility, and transformation of data are meticulously partitioned across these tiers, creating a secure and scalable architecture. Each tier operates on a distinct set of schemas and serves a specific purpose within the vm-constellation ecosystem.

The first tier, designated as **Tier 3**, is an internal, high-security zone corresponding to the **Neural-Resonance-Lab** repository . This tier is solely responsible for ingesting, processing, and storing the most granular forms of BCI data. Here, raw signals from various modalities such as electroencephalography (EEG), heart rate variability (HRV), webcam-based affect detection, and other biosensors are captured [32] [37]. These raw streams contain session IDs, per-sample timestamps, channel arrays, and other fine-grained data points essential for initial signal processing and algorithm development . The critical design principle of this tier is isolation; all data and artifacts produced here, including internal schemas like `bci-telemetry-raw-v1.json` and `bci-telemetry-redacted-v1.json`, are quarantined and never leave the Tier 3 environment or appear in public-facing repositories like the Dead-Ledger . This isolation is a primary security measure, preventing any potential leakage of sensitive biometric data or PII that could be re-identified . By containing the rawest form of data within a single, trusted domain, the framework ensures that privacy risks are managed at their source, adhering to principles of data minimization and privacy-by-design.

Moving up the stack, the second tier involves the **Redaction and Aggregation Layer**, which encompasses repositories such as Redacted-Chronicles and Liminal-Continuum . Data flows from Tier 3 to this tier for processing that includes redaction, aggregation, and summarization. The output of this stage is represented by the `bci-telemetry-summary-v1.json` schema . This schema contains coarse aggregates derived from the raw data, such as mean fear index per event, time spent in each intensity mode bucket, and metric deltas against invariant caps like UEC, ARR, and CDL . Crucially, this process strips away any direct identifiers, such as per-session IDs or timestamps detailed enough

for re-identification, leaving only anonymized statistical summaries . This transformation is fundamental to making BCI data usable for broader system functions, such as ranking seeds, informing policy decisions, or training machine learning models, without exposing the underlying raw signals that could pose privacy risks. This tier acts as a trusted intermediary, sanitizing the data before it enters the more widely distributed parts of the constellation, thereby enabling pattern-level analysis while cryptographically ruling out contamination from raw device data .

At the highest level, the third tier is the **Governance and Execution Layer**, which corresponds to the Dead-Ledger, Orchestrator, and runtime environments . At this level, the system interacts with highly abstracted representations of BCI state, far removed from the complexities of raw physiology. The primary interface here is the `bcistate` proof envelope, an extension of the master `zkpproofv1.json` schema . The public inputs of this ZKP are not raw channels but small, derived metrics such as a `fearband` label or an `overloadflag` . Similarly, BCI seeds are governed by `bciseed.v1.json` profiles that map these high-level states to concrete operational parameters, such as maximum allowed `safetytier` or `intensityband` . This ensures that the core game logic, director scripts, and policy engines operate on a small, well-defined set of discrete states —namely `UNDERSTIMULATED`, `OPTIMAL_STRESS`, and `OVERWHELMED`. This abstraction is what enables true device-agnosticism; the runtime logic does not care *how* the fear index was calculated from EEG alpha/beta waves or a webcam's facial expression classifier; it only cares that a valid proof attests to the resulting state . This tiered model creates a clear boundary between the scientific work of BCI signal processing (owned by Neural-Resonance-Lab) and the governance of horror content generation (owned by Dead-Ledger). It allows the former to innovate and experiment freely with new mapping algorithms and sensor modalities without breaking the stable contracts that the latter depends on, with the `docs/bci-mapping-experiments.md` file serving as the official documentation bridge for these internal methodologies .

Data Tier	Repository/Owner	Primary Function	Key Artifacts/Schemas	Visibility
Tier 3 (Lab-Internal)	Neural-Resonance-Lab	Ingestion and raw signal processing	`bci-telemetry-raw-v1.json`, `bci-payload-v1.json`, `adapter-*.lua`	Internal only
Tier 2 (Redaction/Aggregation)	Redacted-Chronicles, Liminal-Continuum	Data sanitization, aggregation, and summarization	`bci-telemetry-redacted-v1.json`, `bci-telemetry-summary-v1.json`	Restricted (Tier 2/Tier 1)
Tier 1 (Governance/Execution)	Dead-Ledger, Orchestrator, Runtime	Policy enforcement, entitlement control, game logic	`bcistate` ZKP envelope, `bciseed.v1.json`, ledger attestations	System-wide

This structured hierarchy provides a robust solution to the challenge of integrating BCI technology into a complex, multi-repository software ecosystem. It compartmentalizes

risk, standardizes data flow, and creates clear contractual boundaries between different functional domains. By ensuring that only the necessary, sanitized, and abstracted information is passed upwards through the layers, the framework successfully balances the demands of data-driven personalization with the paramount needs for user privacy, data security, and system stability.

Cryptographic Contracts for Safety and Creative Modulation

To fulfill the research goal of enforcing safety and comfort as hard requirements, the framework employs a sophisticated cryptographic protocol based on Zero-Knowledge Proofs (ZKPs) and signed attestation schemas. This system moves beyond simple numerical checks, creating verifiable commitments that can be cryptographically audited and automatically enforced by the `Policy.DeadLedger` module. This transforms abstract safety goals into a set of machine-verifiable rules, ensuring that no BCI-driven action can proceed if it violates a user's safety profile. Within this rigid safety envelope, the framework also defines a mechanism for creative modulation, allowing for dynamic and engaging horror experiences when the user is in a safe and optimal state. This dual-layer approach satisfies the directive to prioritize safety first, then permit creative variation as a secondary consideration .

The foundation of this protocol is the `zkpproofv1.json` schema, which acts as a generic envelope for all types of cryptographic proofs within the vm-constellation . The framework specifies extending this schema to include a dedicated `"prooftype": "bcistate"` . A `bcistate` proof would serve to attest to a specific statement about the user's neurophysiological state, using a unique `statementid` such as `"bcistate.fear-band.in-safe-range.v1"` . The power of this ZKP lies in its ability to prove that the user's derived BCI state (e.g., their current fear index) falls within a predefined, safe range without revealing the underlying raw data or the actual value itself [1] . This preserves user privacy while providing undeniable evidence of compliance with safety constraints. The `publicinputs` field of the proof is explicitly defined to contain only these small, derived metrics, such as a fear band label or an overload flag, and is strictly forbidden from ever containing raw sensor channels .

Complementing the ZKP mechanism is the `bciseed.v1.json` schema, which defines a BCI seed profile . A `bciseed` profile is a declarative rulebook that maps specific BCI states to concrete operational consequences. It takes one or more `bciseedprofileid`

values and links them to the allowed ranges of derived BCI metrics, such as `fear_index`, `overload_flag`, and `fatigue_band` . For each profile, it specifies the corresponding maximum `safetytier` and permissible `intensityband` . For example, a `bciseed` profile might stipulate that if the `bcistate` proof indicates the user is in the `OVERWHELMED` mode, the system's `intensityband` must be capped at 'Tense' and no 'Shocking' events may be triggered. This profile becomes a portable, shareable artifact that dictates the behavioral boundaries for any spectral seed or agent that references it .

These two components—the `bcistate` proof and the `bciseed` profile—are brought together through a series of attestation schemas stored on the Dead-Ledger. An attestation for a `SpectralSeed` (`ledgerentry.spectral-seed.attestation.v1.json`) or an `Agent` (`ledgerentry.agent.attestation.v1.json`) would reference both a unique `seedid` and an optional `bciseedprofileid` . When a runtime component, such as the `Director.` module, attempts to activate a seed or agent, it must present a valid `bcistate` proof to the `Policy.DeadLedger` module. The policy module then verifies that the provided proof satisfies the constraints defined in the referenced `bciseedprofileid` . If the proof is valid and the state is deemed safe, the policy grants access or execution rights. If the proof is invalid or fails to meet the safety criteria (e.g., the user is `OVERWHELMED`), the request is denied, effectively enforcing the safety constraint as a hard requirement. This entire process is automated and trustless, relying on cryptographic verification rather than manual oversight.

Within this rigidly enforced safety framework, creative modulation is enabled as a secondary, constrained capability. This is achieved by defining wider ranges of acceptable `intensityband` values within the `bciseed` profile for the `OPTIMAL_STRESS` state. For instance, a `bciseed` profile might specify that when the `bcistate` proof shows the user is in `OPTIMAL_STRESS`, the `intensityband` can vary between 'Eerie' and 'Tense'. This provides the `Director.` script with a degree of freedom to dynamically choose the level of horror response based on subtle shifts in the user's continuous metrics, such as a rising `arousal` value or a slight increase in the smoothed `fear_index` . However, this modulation remains firmly bounded by the safety envelopes. The system will automatically trigger a downshift if the state approaches `OVERWHELMED`, and it will cease all creative modulation once the user enters that state . The human-readable `deadledger-policy-profiles.md` document provides the overarching entitlement matrix, while the ZKP envelopes and schemas provide the automated enforcement mechanism, ensuring that all generated content remains compliant with the broader Dead-Ledger governance model . This cryptographic contract system thus provides a robust, auditable, and automated way to balance the competing demands of user safety and engaging, adaptive horror storytelling.

A Canonical Runtime Interface for Cross-Device Consistency

To achieve the goal of "cross-device consistency," the framework mandates the creation of a strictly defined, canonical runtime interface that abstracts away the specifics of any particular BCI hardware. This interface ensures that the core game logic, director scripts, and policy engines perceive the user's neuro-state in a uniform manner, regardless of whether the underlying data originates from a high-end EEG headset, a consumer-grade webcam affect detector, or another novel sensor modality. This abstraction is critical for building a scalable and future-proof system, as it prevents device-specific logic from leaking into the main codebase and allows the platform to seamlessly integrate new BCI technologies without requiring changes to the core horror generation logic. The interface is built around a set of standardized schemas and a narrowly focused Lua API surface .

The bedrock of this interface is the **canonical BCI Payload Schema**, specified in `schemas/bci-payload-v1.json` . All device adapters, irrespective of vendor or sensor type, are required to normalize their incoming data stream into this single, shared structure before passing it to the core runtime module . This payload is a conceptual object containing several key fields:

- `session_id`: A string scoped to the Tier-3 lab environment to avoid leaking PII.
- `t`: A float representing seconds since the session start, used for time-based correlation.
- `region_id` and `event_id`: Optional strings that allow the payload to be correlated with specific in-game regions or story events.
- Normalized Derived Metrics: These are the most critical fields. They include `fear_index` (a normalized float in [0.0, 1.0]), `arousal` ([-1.0, 1.0]), `valence` ([-1.0, 1.0]), `source` (a coarse modality label like "eeg" or "face"), and `confidence` (a float in [0.0, 1.0] indicating the reliability of the sample) .

This standardization is the fundamental enabler of device-agnosticism. While the internal mapping from raw channels to these normalized metrics is lab-owned and subject to change, the external shape and meaning of the payload remain constant, providing a stable contract for the rest of the system .

This payload is processed by the **Core `BCI.` Module**, located in `engine/engine-bci-adapter.lua` . This module acts as the single source of truth for all BCI state within the runtime. Its primary function is to expose a stable `BCI.` namespace that game logic can call. Device adapters interact with the runtime exclusively by calling a single entry point:

`BCI.update_from_payload(payload)` . This function accepts the normalized payload, updates the internal state of the core module, applies any necessary smoothing or filtering, and recomputes the final, discrete intensity mode. The core module then exposes this computed state through a set of stable getter functions: `BCI.get_intensity_mode()`, `BCI.get_fear_index()`, `BCI.get_arousal_valence()`, and `BCI.get_overload_flag()` . By centralizing all BCI state logic within this single module, the framework isolates the complexity of neuro-state estimation from the simpler task of reading a device, ensuring that the perception of the user's state is consistent and predictable.

The discrete nature of the BCI state is further codified through the definition of **Intensity Modes**. The framework formalizes three primary modes—`UNDERSTIMULATED`, `OPTIMAL_STRESS`, and `OVERWHELMED`—as a deterministic function of the payload's `fear_index` and `confidence` scores . The v1 specification provides explicit rules for this mapping, such as: `fear_index <= 0.2` results in `UNDERSTIMULATED`, `0.2 < fear_index < 0.8` results in `OPTIMAL_STRESS`, and `fear_index >= 0.8` results in `OVERWHELMED` . These rules are designed to be unambiguous and easy to implement. Any deviation from these canonical rules, such as implementing a different smoothing algorithm or changing the thresholds, constitutes a breaking change that requires an explicit version bump to the adapter contract and the associated schemas . This ensures that all adapters produce a consistent interpretation of the user's state, fulfilling the cross-device consistency requirement .

Finally, the entire system is supported by a sandboxed **Device Adapter Abstraction**. Device-specific implementations reside in files like `devices/bci/adapter-*.lua` . Their sole responsibility is to manage the connection to a specific hardware or software source (e.g., initializing an SDK, opening a webcam pipeline) and translating its native data format into the canonical payload schema. These adapters are strictly forbidden from calling game logic, modifying invariants or entertainment metrics directly, or exposing any device-specific data to higher-level schemas . This sandboxing ensures that the complexity of managing diverse hardware is contained within these thin modules and does not pollute the main codebase. This strict contract makes all BCI devices plug-compatible from the perspective of the horror runtime, allowing AI-driven code generation to target a stable, abstracted interface rather than writing device-specific branching logic .

Telemetry Pipelines and File Structures for Governance and Analysis

For the vm-constellation to function as a coherent, governable system, a robust and standardized telemetry pipeline is essential. This pipeline must carry data from the raw physiological signals captured by BCI devices through multiple stages of processing, redaction, and aggregation, ultimately producing the summarized records needed for governance, analysis, and policy enforcement. This section details the file structures, naming conventions, and schema relationships that constitute this pipeline, ensuring internal consistency and enabling end-to-end traceability from a raw signal to a ledger-attested outcome. The discipline in file naming and hex-code patterns is critical for maintaining system integrity and allowing automated tools to verify compliance .

The file structure for BCI-related assets follows a clear, versioned, and type-specific convention, primarily centered within the **Neural-Resonance-Lab** repository. This structure keeps related files logically grouped and easily discoverable.

Directory/File Path	Type	Purpose
`schemas/bci-payload-v1.json`	Schema	Defines the canonical shape of a single normalized BCI sample.
`schemas/bci-telemetry-raw-v1.json`	Schema	Defines the structure for Tier-3 raw telemetry records containing session IDs and channel arrays.
`schemas/bci-telemetry-redacted-v1.json`	Schema	Defines the structure for Tier-2 redacted telemetry, stripped of PII and fine-grained details.
`schemas/bci-telemetry-summary-v1.json`	Schema	Defines the structure for Tier-2/1-safe aggregated summaries used by the orchestrator and Dead-Ledger.
`schemas/bciseed.v1.json`	Schema	Defines the contract for a BCI seed profile, mapping BCI states to safety tiers and intensity bands.
`engine/engine-bci-adapter.lua`	Module	Core runtime module that implements the `BCI.` API and state computation logic.
`engine/engine-bci-sim.lua`	Module	Synthetic/replay adapter for headless testing and simulations.
`devices/bci/adapter-*.lua`	Module	Thin, device-specific modules that translate vendor data into the canonical payload.
`docs/bci-device-adapter-contract.md`	Documentation	Formalizes the contract for all BCI device adapters, defining required functions and behaviors.
`docs/bci-telemetry-pipeline.md`	Documentation	Describes the ingestion, redaction, and summarization pipelines for BCI telemetry.

This structure ensures that every component has a designated place and that dependencies are explicit. The use of semantic versioning (e.g., `-v1.json`) in filenames

is a critical convention that allows for future evolution of schemas without breaking backward compatibility .

The telemetry pipeline itself is a multi-stage process that mirrors the tiered data abstraction model. At the **Tier 3 Lab-Internal Stage**, raw data from a BCI device is captured and logged into files that conform to the `bci-telemetry-raw-v1.json` schema. These files contain the least amount of processing, preserving the original timestamp, session ID, and channel data for deep analysis and algorithm refinement . To ensure data integrity and traceability, every BCI-related artifact that crosses repository boundaries—from telemetry summaries to `bciseed` profiles—is validated against its corresponding Tier 1 schema and then cryptographically signed using a Hardware Security Module (HSM) or Key Management Service (KMS) . This process produces a descriptor file containing a canonical hash of the artifact, providing a verifiable proof of its origin and content .

As data moves to the **Tier 2 Redaction and Aggregation Stage**, the raw telemetry is processed. A pipeline consumes the raw files and generates redacted versions that conform to `bci-telemetry-redacted-v1.json`. This step involves quantizing timestamps, removing or obfuscating session identifiers, and potentially coarse-graining the BCI metrics to prevent re-identification . Finally, at the **Tier 2/1 Governance Stage**, the redacted telemetry is aggregated into the summary format defined by `bci-telemetry-summary-v1.json`. This final schema is the only BCI-linked format intended for consumption by other repos, such as Liminal-Continuum, the orchestrator, and the Dead-Ledger . This summary record is rich with metadata, including the `agent_id`, `event_id`, snapshot of `region_bands`, mean metric deltas (ΔUEC, ΔARR, etc.), time fractions spent in each intensity mode, and key safety-relevant maxima like `max_fear_index` . Crucially, all telemetry files at every stage should include a `schema_ref` field that points back to the relevant BCI telemetry schema, allowing CI systems to perform automated validation of the data structure .

This disciplined approach to file naming and pattern management extends to all identifiers within the system. Ledger entries, such as those for bundles, agents, and seeds, must adhere to a strict `deadledgerref` pattern like `DLN.seed.<namespace>.<slug>.v1.0.0` . Hash fields across all schemas should be standardized to a fixed-length uppercase or lowercase hexadecimal string, reflecting a digest algorithm like SHA-256 . This creates a system of opaque, stable identifiers that are meaningful only when resolved through the shared schemas and registries, rather than "magic numbers" that encode behavior directly . By anchoring the meaning of every hex-coded ID in a typed, versioned schema, the system guarantees internal consistency and makes it possible to revoke or supersede any problematic pattern without disrupting

active integrations or live device routing . This comprehensive pipeline and structural discipline provide the necessary infrastructure for the vm-constellation to govern BCI-enabled content safely and effectively.

Implementation Blueprint for BCI Device Adapters

To translate the theoretical framework into practical, deployable code, a detailed implementation blueprint is required. This blueprint focuses on the `docs/bci-device-adapter-contract.md` document, which serves as the definitive specification for all BCI device integrations . It defines the responsibilities of device adapters, the canonical payload they must produce, the core runtime module they interact with, and the rules governing the conversion of raw data into actionable neuro-states. Adherence to this contract ensures that all BCI devices are plug-compatible, device-agnostic, and fully compliant with the vm-constellation's safety and consistency requirements.

The core of the implementation is the **Normalized BCI Payload Shape**, defined by the `schemas/bci-payload-v1.json` schema . Every device adapter, regardless of its underlying technology, must produce a Lua table or JSON object that conforms to this structure. The conceptual fields are as follows:

- `session_id`: A string scoped to the local lab environment, used for correlating data within a single run but not exported externally .
- `t`: A numeric value representing the timestamp in seconds since the beginning of the session, used for time-series analysis .
- `region_id` and `event_id`: Optional strings that allow the payload to be linked to specific game elements, populated by the adapter when known .
- `fear_index`: A float normalized to the range [0.0, 1.0], representing the computed level of fear .
- `arousal` and `valence`: Floats normalized to [-1.0, 1.0], representing the dimensional emotional state .
- `source`: A string enum (e.g., `"eeg"`, `"face"`, `"mixed"`) indicating the modality of the input data, not a specific hardware model .
- `confidence`: A float normalized to [0.0, 1.0] that expresses the algorithm's confidence in the validity of the current sample .

Device adapters are free to implement their own internal logic for mapping raw sensor readings to this payload shape. For example, an EEG adapter might compute `fear_index` from the ratio of high-beta to low-alpha waves, while a webcam affect

detector would use a pre-trained model's output probabilities . The exact mapping is documented internally but must not alter the external payload ranges or semantics.

The interaction between the adapter and the core engine is mediated by the **Core `BCI.` Module** (`engine/engine-bci-adapter.lua`). This module is the single source of truth for BCI state. Its primary contract is a set of functions that adapters must call. The most important of these is `BCI.update_from_payload(payload)`, which takes a single, normalized payload and updates the internal state . This core module is also responsible for implementing the Intensity Mode Rules. For the initial pilot, these rules are deterministic:

- If `fear_index <= 0.2` and `confidence >= 0.5`, the mode is `UNDERSTIMULATED`.
- If `0.2 < fear_index < 0.8` and `confidence >= 0.5`, the mode is `OPTIMAL_STRESS`.
- If `fear_index >= 0.8` and `confidence >= 0.5`, the mode is `OVERWHELMED`.
- If `confidence < 0.5`, the adapter should carry forward the last known mode, defaulting to `OPTIMAL_STRESS` if none exists .

These rules are the basis for all runtime decisions, ensuring that the translation from a continuous physiological signal to a discrete, actionable state is consistent across all devices.

Device adapters themselves are implemented as thin, isolated modules following a common pattern. The `docs/bci-device-adapter-contract.md` specifies a skeleton that each adapter module should return. This module should provide `start()`, `stop()`, and `update(dt, context)` functions . The `start()` function initializes the device connection, `stop()` tears it down, and `update()` is called by the main game loop. Inside `update()`, the adapter reads its device, converts the raw data into the canonical payload, and calls `BCI.update_from_payload(payload)` . This pattern ensures that adapters are self-contained and do not have side effects on the rest of the game engine. They are explicitly forbidden from calling game logic, directors, or story modules directly .

A critical component for robust development and testing is the **Headless Simulation Harness**. The repository must include `engine/engine-bci-sim.lua`, a module designed to replay synthetic or recorded payload sequences into the `BCI.` core module . This allows developers to run repeatable experiments in a shared testbed without needing physical devices. A simulation script would be a list of payload dictionaries, and the `engine-bci-sim.lua` module would iterate through this list, injecting the

payloads into `BCI.update_from_payload` at appropriate time steps . This capability is essential for CI testing, allowing automated checks to assert that intensity modes behave as expected under various synthetic scenarios and that overload flags are triggered correctly . The implementation checklist for any new BCI integration includes choosing a filename, implementing the adapter skeleton, registering it in test harnesses, hooking it into the telemetry pipeline, and running basic validation experiments using the simulation module . By following this blueprint, developers can ensure that their BCI integrations are compliant, reliable, and ready for production use.

Synthesis and Strategic Recommendations for vm-constellation Integration

The research framework presented in this report provides a comprehensive and principled architecture for integrating Brain-Computer Interface (BCI) device adapters into the Horror$Place vm-constellation. It directly addresses the core research goal by establishing a disciplined pathway that ensures compatibility with existing runtime modules, enforces safety and comfort as hard requirements, permits creative modulation as a secondary layer, and maintains cross-device consistency through a strict adherence to canonical contracts. The framework is not a single piece of code but a multi-layered system of contracts, abstractions, and cryptographic proofs that govern the flow of neuro-state information from its raw capture to its use in generating adaptive horror content. This synthesis highlights how the framework achieves these objectives and concludes with strategic recommendations for its implementation.

The framework's success hinges on its three-pillar architecture. First, the **Tiered Data Abstraction Model** creates a secure and scalable data pipeline by quarantining raw, sensitive biometric data at the Tier-3 Neural-Resonance-Lab level while progressively transforming it into standardized, commitment-friendly abstractions at higher tiers . This ensures that the privacy-preserving principles of data minimization are upheld, preventing raw signals from ever reaching the governance and execution layers where they could pose a security risk . Second, the **Cryptographic Contracts for Safety and Modulation** provide a verifiable and automated enforcement mechanism for user safety. By combining `bcistate` Zero-Knowledge Proofs with `bciseed` profiles, the system can cryptographically guarantee that a user's neuro-state complies with predefined safety constraints before allowing any action that might cause distress . This directly translates the directive for safety to be a hard requirement into a machine-enforceable rule. Within this safety envelope, the framework allows for creative modulation by defining a wider

range of acceptable behaviors for the `OPTIMAL_STRESS` state, enabling dynamic and engaging content without compromising user comfort . Third, the **Canonical Runtime Interface** ensures device-agnosticism and cross-device consistency. By mandating that all device adapters project their data into a single canonical payload schema and adhere to deterministic intensity mode rules, the framework insulates the core game logic from hardware-specific variations, making the system adaptable to new BCI technologies without modification .

In summary, this framework successfully guides the development of BCI device adapters by providing a clear, coherent, and safe integration path. It ensures compatibility with existing modules like `Policy.DeadLedger` by expressing all BCI-related logic through established APIs rather than creating parallel interfaces . It enforces safety as a non-negotiable constraint through cryptographic proofs and automated system responses, such as automatic downshifts during `OVERWHELMED` states . It allows for creative modulation by operating within the flexible parameters permitted by `bciseed` profiles when the user is in the `OPTIMAL_STRESS` band. Finally, it maintains cross-device consistency by strictly adhering to the v1 intensity mode rules and canonical payload schema, allowing for lab-specific deviations only under clearly versioned contracts .

To move from this conceptual framework to a deployed system, the following strategic recommendations are proposed:

1. **Formalize and Adopt the Core Contracts:** The `docs/bci-device-adapter-contract.md` document should be finalized, peer-reviewed, and adopted as the official specification for all BCI integrations across the vm-constellation . This document provides the necessary detail for developers to build compliant adapters.

2. **Implement the Tiered Telemetry Pipelines:** The corresponding Lua modules for handling the `bci-telemetry-raw`, `redacted`, and `summary` pipelines must be developed in the Neural-Resonance-Lab repository. These implementations should be integrated with the CI/CD workflows to include automated schema validation, ensuring data integrity at every stage of the pipeline.

3. **Develop the ZKP Infrastructure:** The cryptographic backbone requires development. This includes setting up the ZK verifier registry (`verifiers.json`) and building the tooling necessary to generate `bcistate` proofs and validate them within the `Policy.DeadLedger` module . Initial experiments should focus on proving simple statements, such as a `fear_index` being below a certain threshold.

4. **Create Concrete Seed Examples:** To demonstrate the practical application of the framework, concrete examples of `bciseed.v1.json` profiles should be created for different types of horror experiences (e.g., "Slow-Burn Psychological Thriller Seed," "High-Impact Jump-Scares Seed"). These examples will illustrate how safety caps and creative modulation parameters are configured for real-world use cases.

By following these steps, the Horror$Place project can systematically build out the BCI integration capabilities defined by this research, laying a solid foundation for a new generation of safe, personalized, and deeply adaptive horror experiences.

Reference

1. [PDF] On the Trustworthiness of Generative Foundation Models - arXiv https://arxiv.org/pdf/2502.14296
2. (PDF) Future of Res in Mgmt n AI by DeepScience, Full book, final ... https://www.researchgate.net/publication/397743810_Future_of_Res_in_Mgmt_n_AI_by_DeepScience_Full_book_final_Nov-2025
3. Homing in on consciousness in the nervous system https://neuroscape.ucsf.edu/wp-content/uploads/homing-in-on-consciousness-in-the-nervous-system-an-action-based-synthesis.pdf
4. (PDF) The Extended Mind & Body in Extended Realities https://www.researchgate.net/publication/384098624_The_Extended_Mind_Body_in_Extended_Realities_A_Scoping_Review_of_XR_Applications_and_Risks_in_the_Metaverse
5. Download book PDF - Springer Nature https://link.springer.com/content/pdf/10.1007/978-3-642-21675-6.pdf
6. [PDF] Mapping the World of Anglo-American Studies at the Turn of the ... https://www.researchgate.net/profile/Sabina-Osmanovic/publication/377075170_Dealing_with_Evanescence_The_Motif_of_Old_Age_and_the_Crisis_of_Meaning_in_Philip_Roths_The_Dying_Animal/links/6594167c3c472d2e8eaaf8fe/Dealing-with-Evanescence-The-Motif-of-Old-Age-and-the-Crisis-of-Meaning-in-Philip-Roth-s-The-Dying-Animal.pdf
7. Adapter Modules | SAP Help Portal https://help.sap.com/docs/migration-guide-po/migration-guide-for-sap-process-orchestration/adapter-modules

8. 333333 23135851162 the 13151942776 of 12997637966 ftp://ftp.cs.princeton.edu/pub/cs226/autocomplete/words-333333.txt
9. (PDF) Scribe Publications mINDSIGHT https://www.academia.edu/22127627/Scribe_Publications_mINDSIGHT
10. Program Committee https://www.researchgate.net/profile/Ryohei-Nakatsu/publication/229060121_Emotion-based_multi-person_interactive_theater_Romeo_and_Juliet_in_Hades/links/543fbed10cf2be1758cfb05e/Emotion-based-multi-person-interactive-theater-Romeo-and-Juliet-in-Hades.pdf
11. MMI-Interaktiv - Nr. 13 - User Experience https://www.researchgate.net/profile/Theo-Held/publication/236524075_Untersuchung_von_Designpraferenzen_mit_Hilfe_von_Skalierungsmethoden/links/0046353b4e18c87933000000/Untersuchung-von-Designpraeferenzen-mit-Hilfe-von-Skalierungsmethoden.pdf
12. 333333 23135851162 the 13151942776 of 12997637966 https://www.cs.princeton.edu/courses/archive/spring18/cos226/assignments/autocomplete/testing/words-333333.txt
13. cmnt_vocab.txt - CMU School of Computer Science https://www.cs.cmu.edu/~ark/blog-data/data/blog_data_v1_0/dk/hbc_data/data/cmnt_vocab.txt
14. Requested packages, organized by age - Debian https://www.debian.org/devel/wnpp/requested_byage
15. Debian -- Prospective packages https://www.debian.org/devel/wnpp/prospective
16. Requested packages - Debian https://www-staging.debian.org/devel/wnpp/requested
17. Chinese teachers' feedback on Korean CFL learners' cohesive ... https://www.academia.edu/31143519/Chinese_teachers_feedback_on_Korean_CFL_learners_cohesive_errors_in_ICSSD_2016_Proceedings
18. Advances in Artificial Intelligence, Software and Systems https://www.scribd.com/document/713632844/Advances-in-Artificial-Intelligence-Software-and-Systems-Tareq-Z-Ahram-Editor-Waldemar-Karwowski-Editor-Jay-1st-Ed-2021-2021-978303
19. glove.6B.100d.txt-vocab.txt https://worksheets.codalab.org/rest/bundles/0xadf98bb30a99476ab56ebff3e462d4fa/contents/blob/glove.6B.100d.txt-vocab.txt
20. [PDF] From Rules to Language Models - ACL Anthology https://aclanthology.org/2025.lm4dh-1.pdf
21. A New AI Research from Anthropic and Thinking Machines Lab ... https://www.marktechpost.com/2025/10/25/a-new-ai-research-from-anthropic-and-thinking-machines-lab-stress-tests-model-specs-and-reveal-character-differences-among-language-models/

22. Quality of Information and Communications Technology https://link.springer.com/content/pdf/10.1007/978-3-030-58793-2.pdf
23. [PDF] journal of the management sciences (jomas) - ResearchGate https://www.researchgate.net/profile/Nonso-Okoye-2/publication/361893083_Microfinance_and_small_scale_enterprise_in_NigeriaAn_empirical_study/links/62cb3f523bbe636e0c538b69/Microfinance-and-small-scale-enterprise-in-NigeriaAn-empirical-study.pdf
24. Game Research Methodshods Lankoski Bjork Etal Web | PDF - Scribd https://www.scribd.com/document/282759507/Game-research-methodshods-Lankoski-Bjork-Etal-Web
25. (PDF) Conference - ICMTCC PROCEEDINGS (1) - Academia.edu https://www.academia.edu/144456455/Conference_ICMTCC_PROCEEDINGS_1_
26. Blog | ACM Interactions https://interactions.acm.org/blog/view/abatmaz@sfu.ca
27. Google 1-gram - CMU School of Computer Science https://www.cs.cmu.edu/~roni/11761/2017_fall_assignments/hw3_stats_google_1gram.txt
28. [PDF] IBM InfoSphere Streams: Harnessing Data in Motion https://www.redbooks.ibm.com/redbooks/pdfs/sg247865.pdf
29. Design, User Experience, and Usability - Springer Nature https://link.springer.com/content/pdf/10.1007/978-3-031-05897-4.pdf
30. Debian -- Requested packages https://www.debian.org/devel/wnpp/requested
31. (PDF) IoT-Based Stress Monitoring Using CNN for HRV-GSR Analysis https://www.researchgate.net/publication/399757973_IoT-Based_Stress_Monitoring_Using_CNN_for_HRV-GSR_Analysis
32. A Review of AI Cloud and Edge Sensors, Methods, and Applications ... https://pmc.ncbi.nlm.nih.gov/articles/PMC9611164/
33. [PDF] Data-driven design in virtual reality: combining user perception with ... https://theses.hal.science/tel-04849713/file/135634_SAUNIER_2024_archivage_final.pdf
34. Foundation of Affective Computing & Interaction - arXiv https://arxiv.org/html/2506.15497v1
35. Digital Cardiovascular Twins, AI Agents, and Sensor Data - MDPI https://www.mdpi.com/1424-8220/25/17/5272
36. A Proposal for a Data-Driven Approach to the Influence of Music on ... https://www.frontiersin.org/journals/cardiovascular-medicine/articles/10.3389/fcvm.2021.699145/full
37. Biometric Systems PDF - Scribd https://www.scribd.com/document/426255987/biometric-systems-pdf

38. Comprehensive Glossary of Telecom | PDF - Scribd https://fr.scribd.com/doc/266101671/Comprehensive-Glossary-of-Telecom